Adaptive Traffic Signal Control Using Deep Q-Networks: A Comparative Study with Rule-Based Method Under Varying Headway Patterns

Prepared by: Saurav Barua

Work plan

❑Testbed

SUMO, Traci

❑Network Description

- ✓3 nos. 4-leg intersection, each located 600m apart.
- ✓3 lanes for each EB and WB approaches and 3 lanes for each NB and SB approaches.
- ✓NB and SB approaches are minor arterials, EB and WB approaches are major arterials.

❑Competitive models

1. Rule-based adaptive signal (baseline)
2. DQN based adaptive signal

❑16 Headways patterns

- Basic- Uniform, Random, Platoon
- Peak hours- Morning peak, Evening peak
- Extreme- Oversaturated, Free-flow, mixed fleet
- Realistic- Urban Arterial, highway merge, incident, bimodal
- Statistical- Log-normal, Gamma, Weibull, Erlang

❑Performance metrics

- ✓Average Queue, Average Speed, Average travel time, Average delay, throughput

Simulation and Performance Evaluation

1. Generate Traffic with Specific Headway Pattern

Creating realistic traffic flow

2. Start SUMO Simulation via TraCI

Launching the simulation

3. Select Action Using Epsilon-Greedy Policy

Choosing actions based on policy

4. For Each Simulation Step (every 5 seconds)

Iterating through simulation steps

5. Calculate Reward Based on Traffic Performance

Evaluating traffic performance

6. Store Experience in Replay Memory

Saving experiences for training

7. Collect Performance Metrics

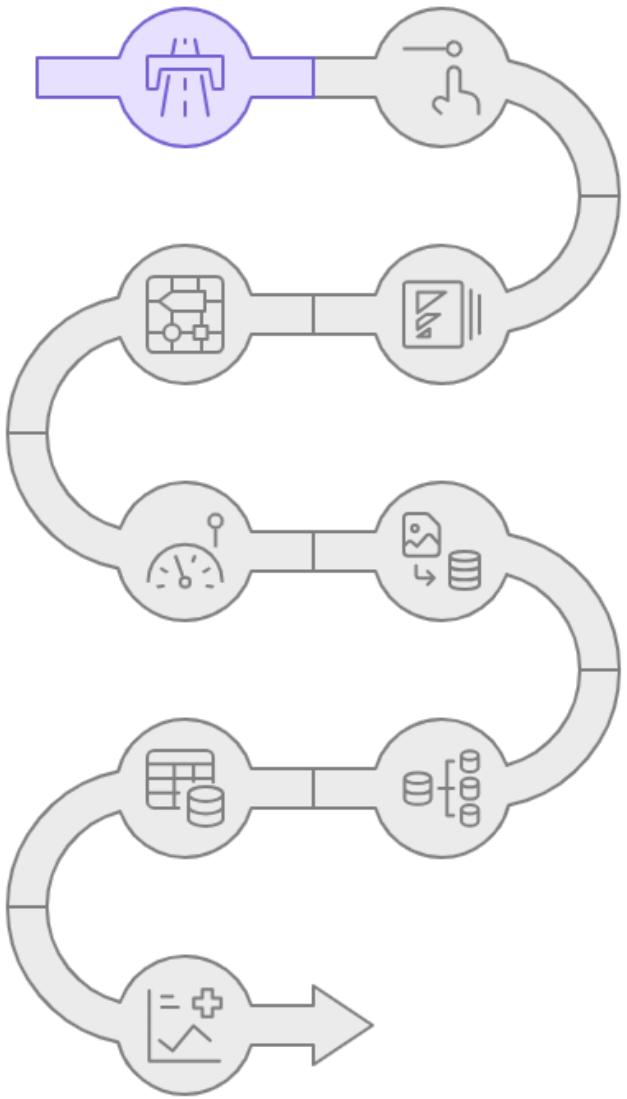
Gathering simulation results

8. Train DQN Networks

Training the deep learning models

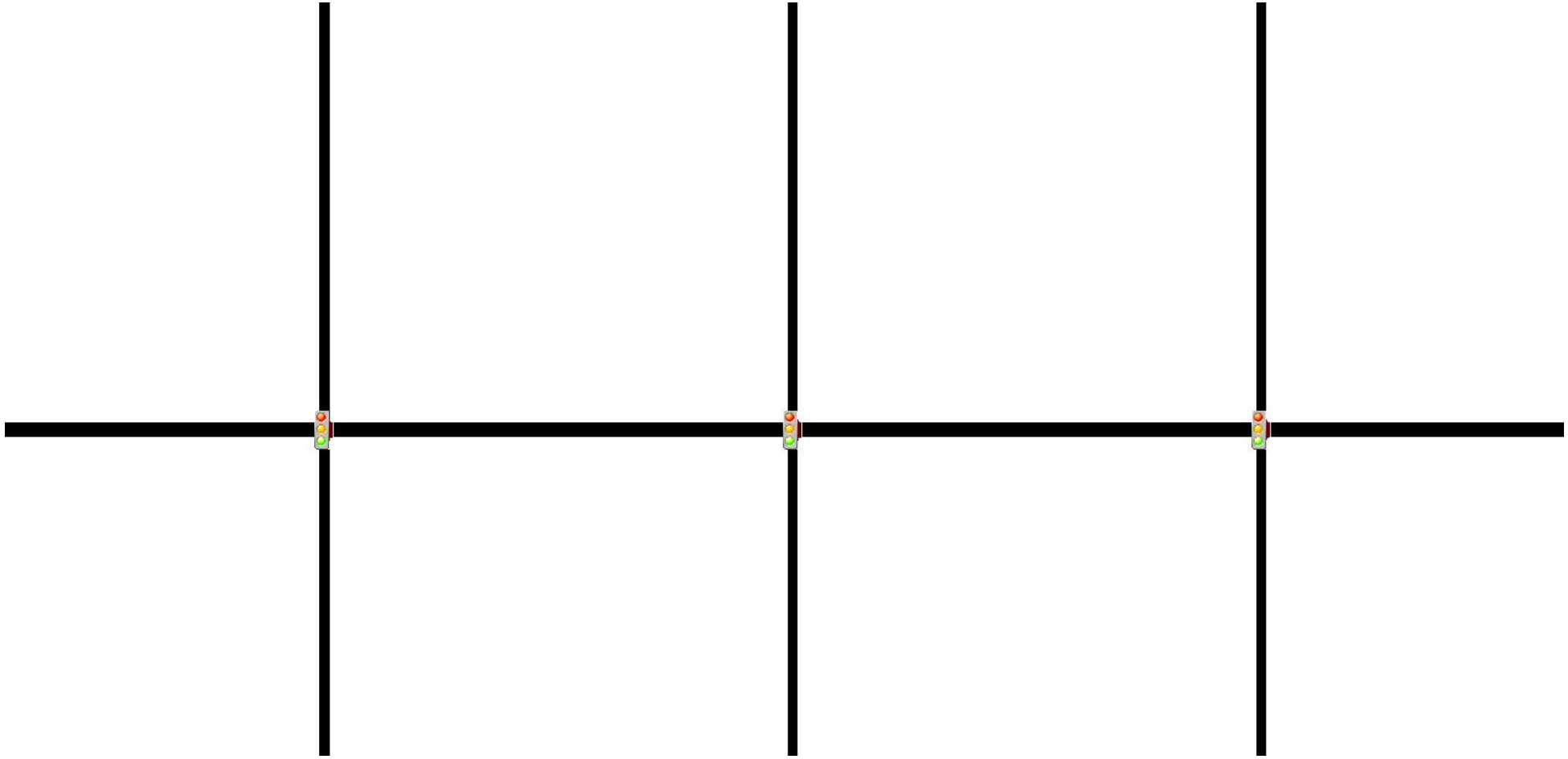
9. Analyze & Compare Results

Evaluating and comparing outcomes



Made with Napkin

Road network in SUMO



A snapshot of road network in SUMO netEdit

Competitive models
1. Rule-based adaptive signal (baseline)

Pseudo code

```
FOR each intersection i in [J1, J2, J3]:
   IF current_phase in [GREEN] AND
      phase_duration >= MIN_GREEN (20s):
     main_queue, cross_queue = get_queues(i)
     should_extend = (phase_is_main AND
                      main_queue > 1.2 * cross_queue) OR
                     (phase_is_cross AND
                      cross_queue > 1.2 * main_queue)
     coordination_override = (cycle_time - offset[i]) % 90 > 80
     IF should_extend AND
       NOT coordination_override AND
       phase_duration < MAX_GREEN (60s):
        extend_current_phase_by(5s)
     ELSE:
        switch_to_next_phase()  # Green→Yellow→Next Green
```

Explanation

1.Monitor each intersection: Continuously check if current green phase has exceeded minimum green time (20s) before allowing changes

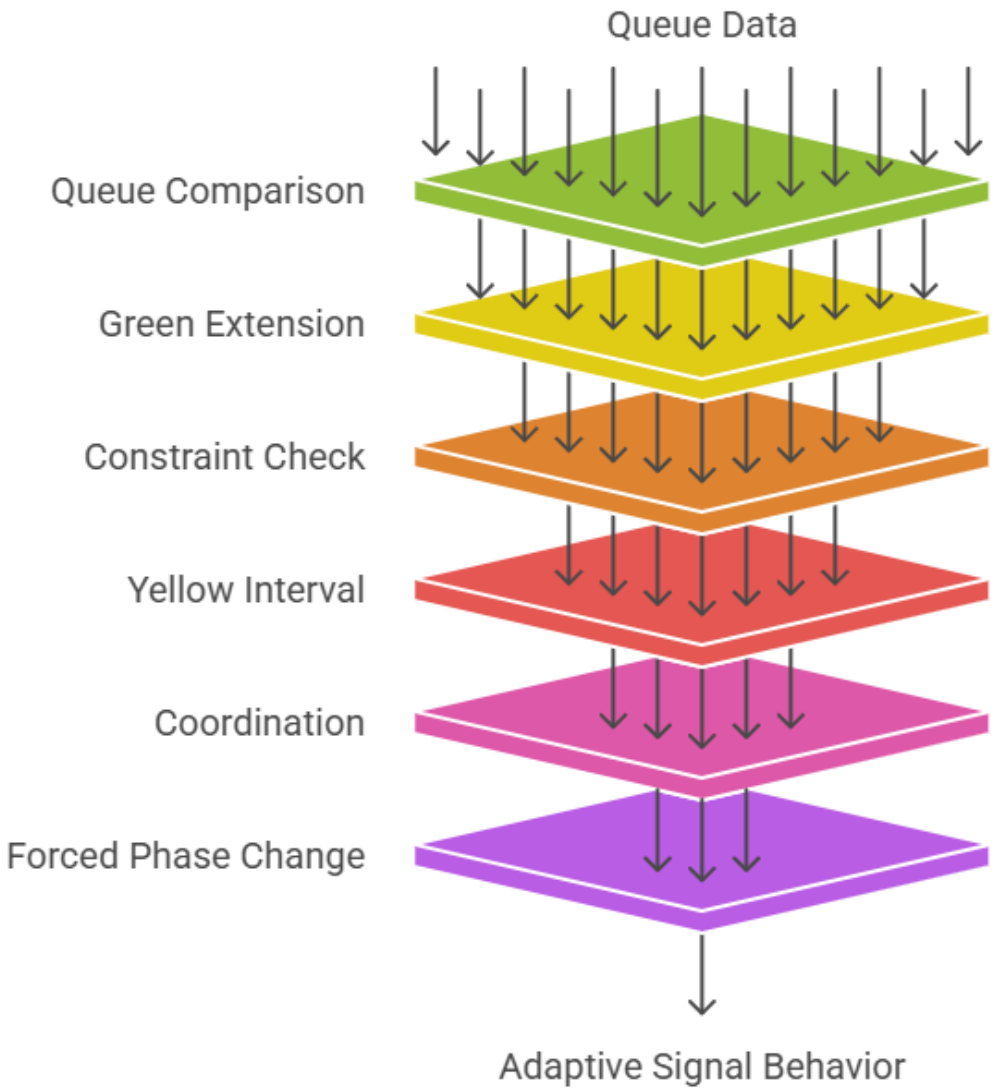
2.Compare approach demands: Calculate vehicle queues on main street vs cross street to determine if current phase should be extended

3.Apply extension logic: Extend green by 5 seconds if current approach has 20% more vehicles than competing approach, up to maximum green (60s)

4.Override for coordination: Force phase termination during last 10 seconds of 90s cycle to maintain green wave offsets between intersections

5.Default transition: Switch to next phase (green→yellow→next green) when extension conditions aren't met or max green reached

Baseline Rule-Based Adaptive Signal Process



Made with Napkin

Competitive models
2. DQN adaptive signal

Pseudo code

```
state = extract_features(queues, speeds,
    waiting_times, phase_duration)
action = select_action(state, epsilon)
    # ε-greedy: explore vs exploit Q-values
execute_action(action)
    # Change phase if different and min green met
reward = -5.0*queue_norm -3.0*delay_norm
    +4.0*speed_norm +0.1*throughput
store_experience(state, action, reward, next_state)
    in replay memory
sample_random_batch from memory
    to train policy network (minimize MSE loss)
update_target_network every 1000 steps
    for stable Q-value targets
decay_epsilon(ε_start=1.0 → ε_end=0.05)
    to shift exploration→exploitation
```

Explanation

1.State extraction: Convert real-time traffic data (queues, speeds, waiting times) into a 36-feature vector representing intersection conditions

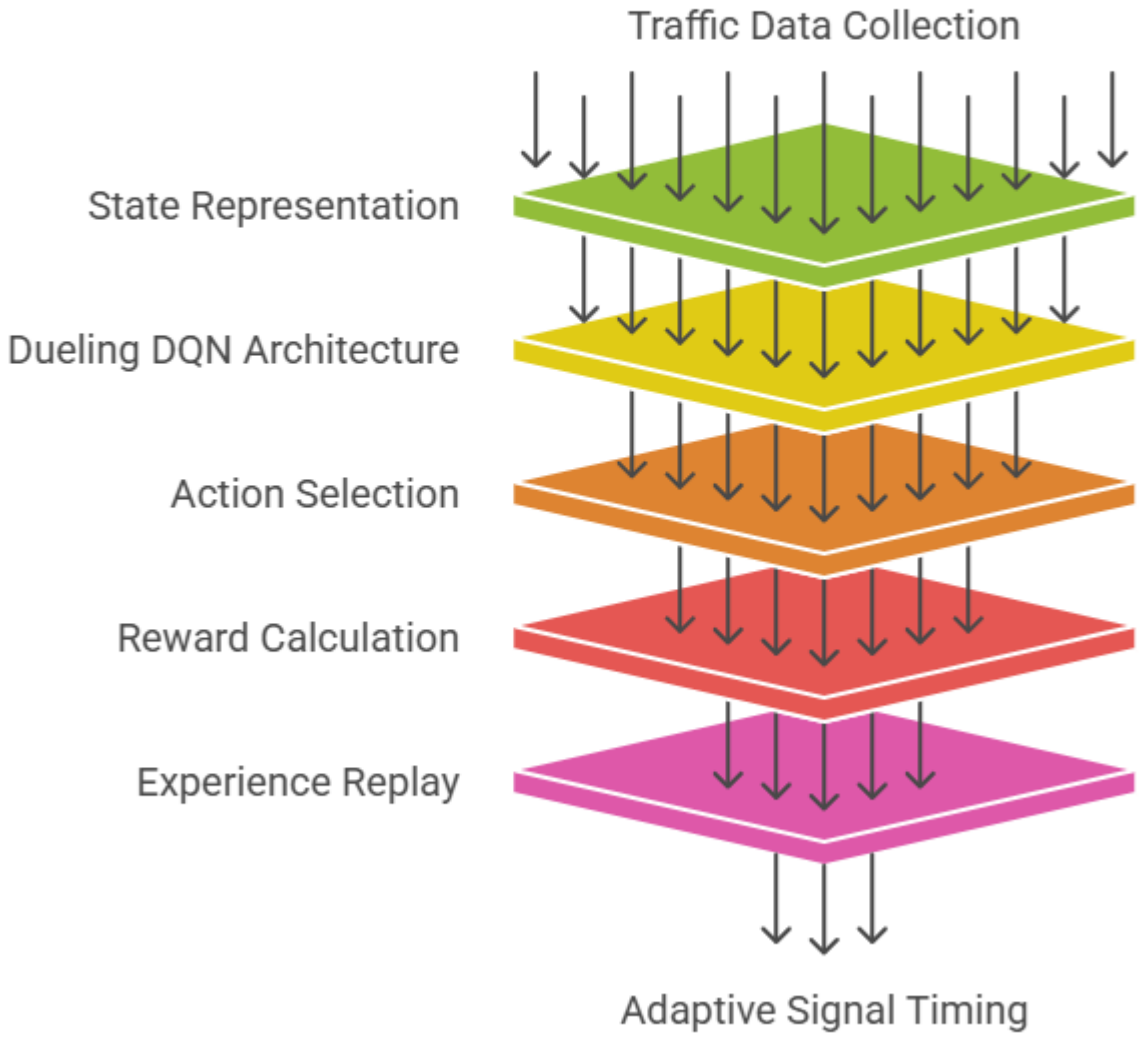
2.Action selection: Use epsilon-greedy policy to balance exploration (random actions) vs exploitation (best Q-value actions from neural network)

3.Reward calculation: Compute weighted score prioritizing queue reduction (-5.0) and speed (+4.0) over throughput (+0.1) to prevent gaming

4.Experience replay: Store transitions in memory and randomly sample batches to train network, breaking temporal correlations

5.Network updates: Train policy network via MSE loss against target network, slowly updating target network for stable learning

DQN Adaptive Traffic Signal Learning Process



Made with Napkin

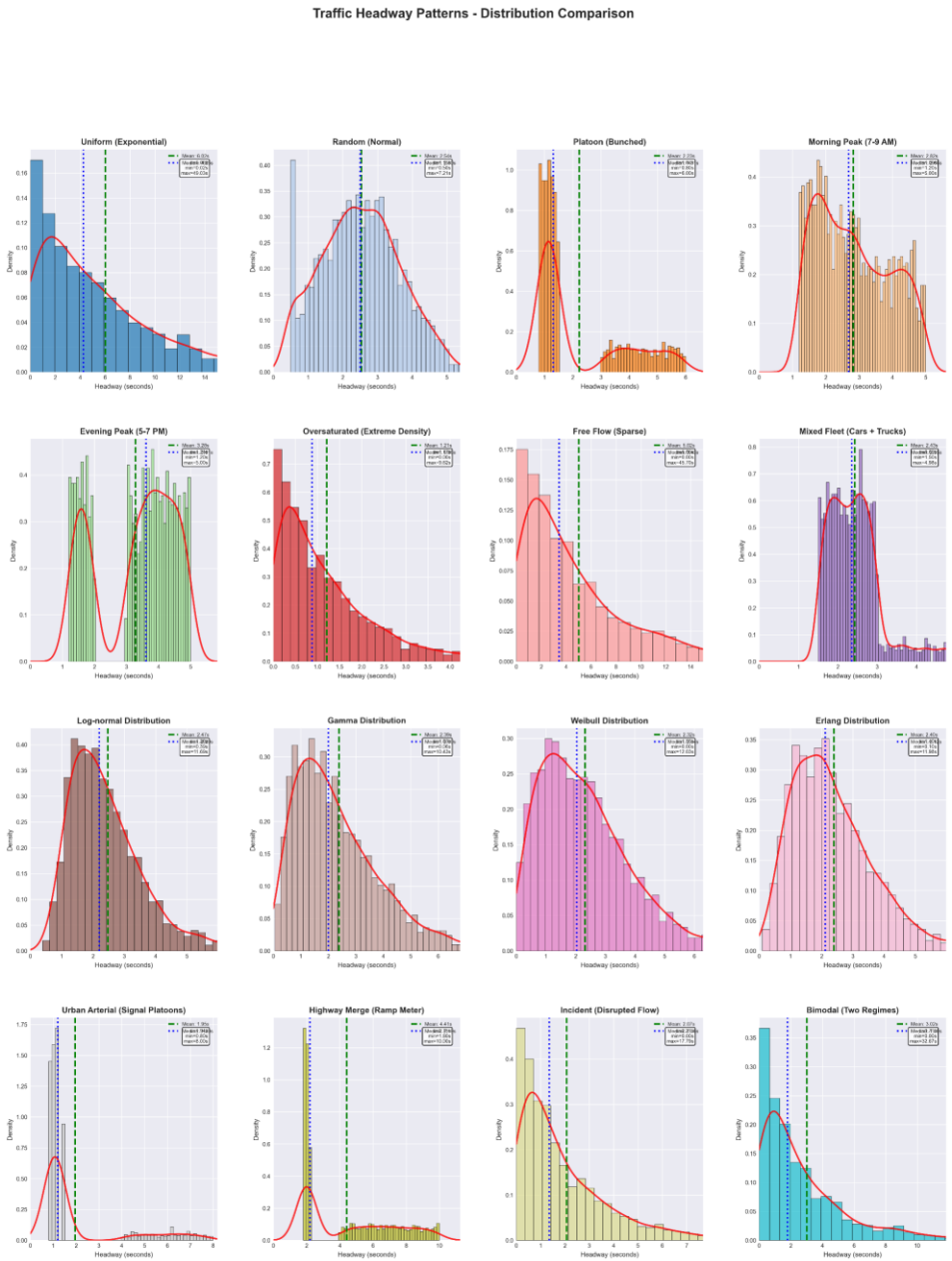
Headway patterns

- Basic Patterns: Uniform (exponential/Poisson process for random arrivals), Random (normal distribution with 2.5s mean), and Platoon (bunched 3-6 vehicles at 0.8-1.5s headways with 3-6s gaps)
- Peak Hour Patterns: Morning Peak (heavy 7-9 AM traffic with 1.2-2.0s headways) and Evening Peak (heavy 5-7 PM traffic) simulating rush hour congestion
- Extreme Conditions: Oversaturated (extremely dense 1.2s headways near capacity), Free-flow (sparse 5.0s headways), and Mixed Fleet (90% cars 1.5-3.0s + 10% trucks 3.0-5.0s)
- Statistical Distributions: Log-normal (positively skewed), Gamma (flexible shape parameter 2.0), Weibull (reliability theory with shape 1.5), and Erlang (sum of 3 exponentials)
- Real-World Patterns: Urban Arterial (signal-induced platoons of 4-10 vehicles), Highway Merge (ramp meter releases of 2-4 cars at 1.8-2.2s), Incident (disrupted flow with 1.2s headways in middle third), and Bimodal (60% congested 1.8s + 40% free-flow 4.5s)

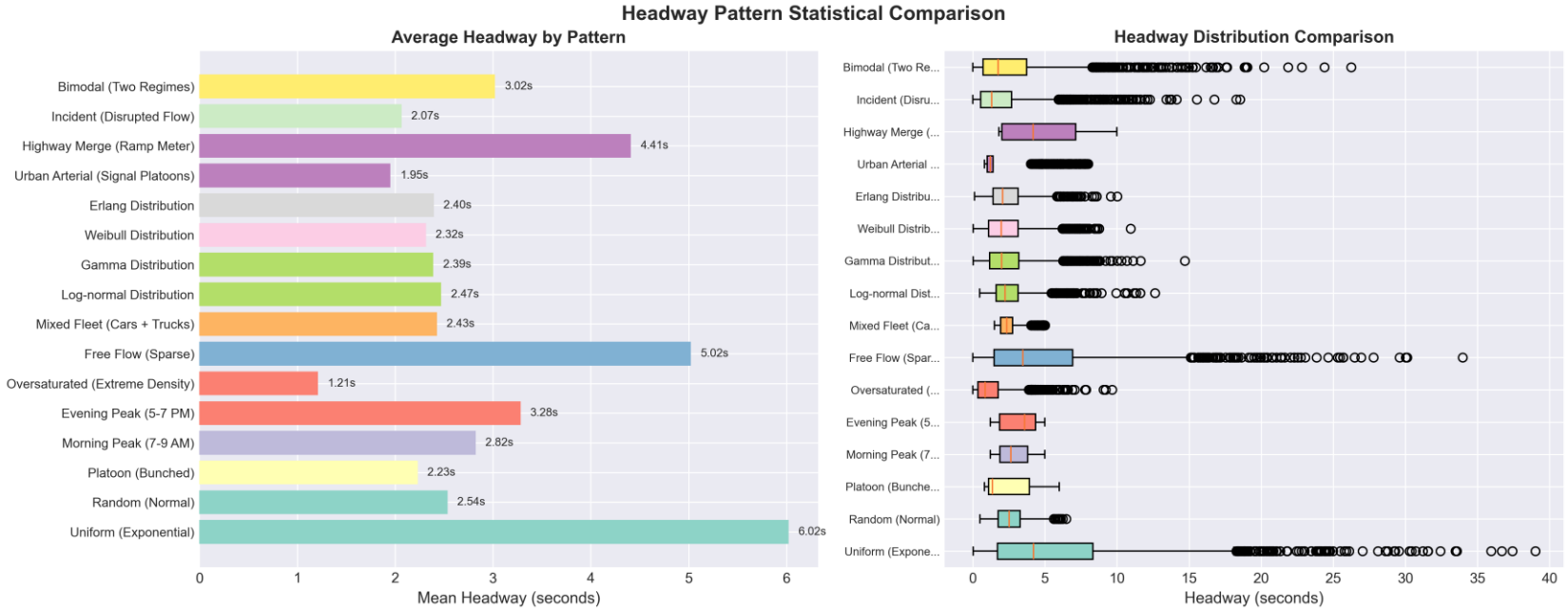
Performance metrics

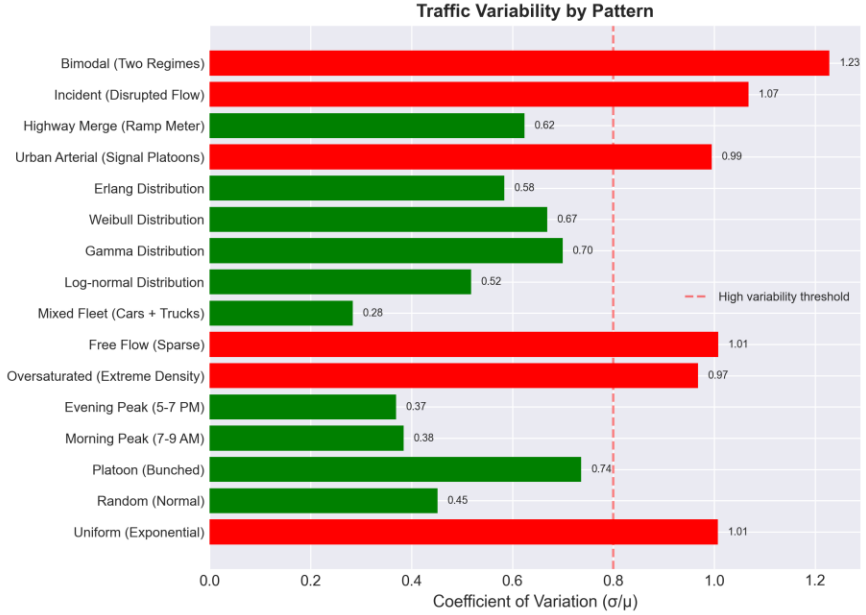
1. Average Queue Length: Sum of vehicles waiting on all lanes (excluding internal intersection lanes) divided by total number of lanes, sampled every 10 seconds throughout simulation and then averaged
2. Average Speed: Mean speed of all active vehicles in network (converted from m/s to km/h), sampled every 10 seconds after warmup period, then averaged across all time steps
3. Average Travel Time: For each completed vehicle, calculate exit time minus entry time, then average across all vehicles that successfully traversed the network during simulation
4. Average Delay: For each completed vehicle, calculate (actual travel time - free-flow travel time) where free-flow time = route length ÷ 13.89 m/s (50 km/h), then average across all vehicles
5. Throughput: Number of vehicles that complete their journey and exit the network, converted to hourly rate: (completed vehicles × 3600) ÷ (simulation time - warmup time

Headway patterns



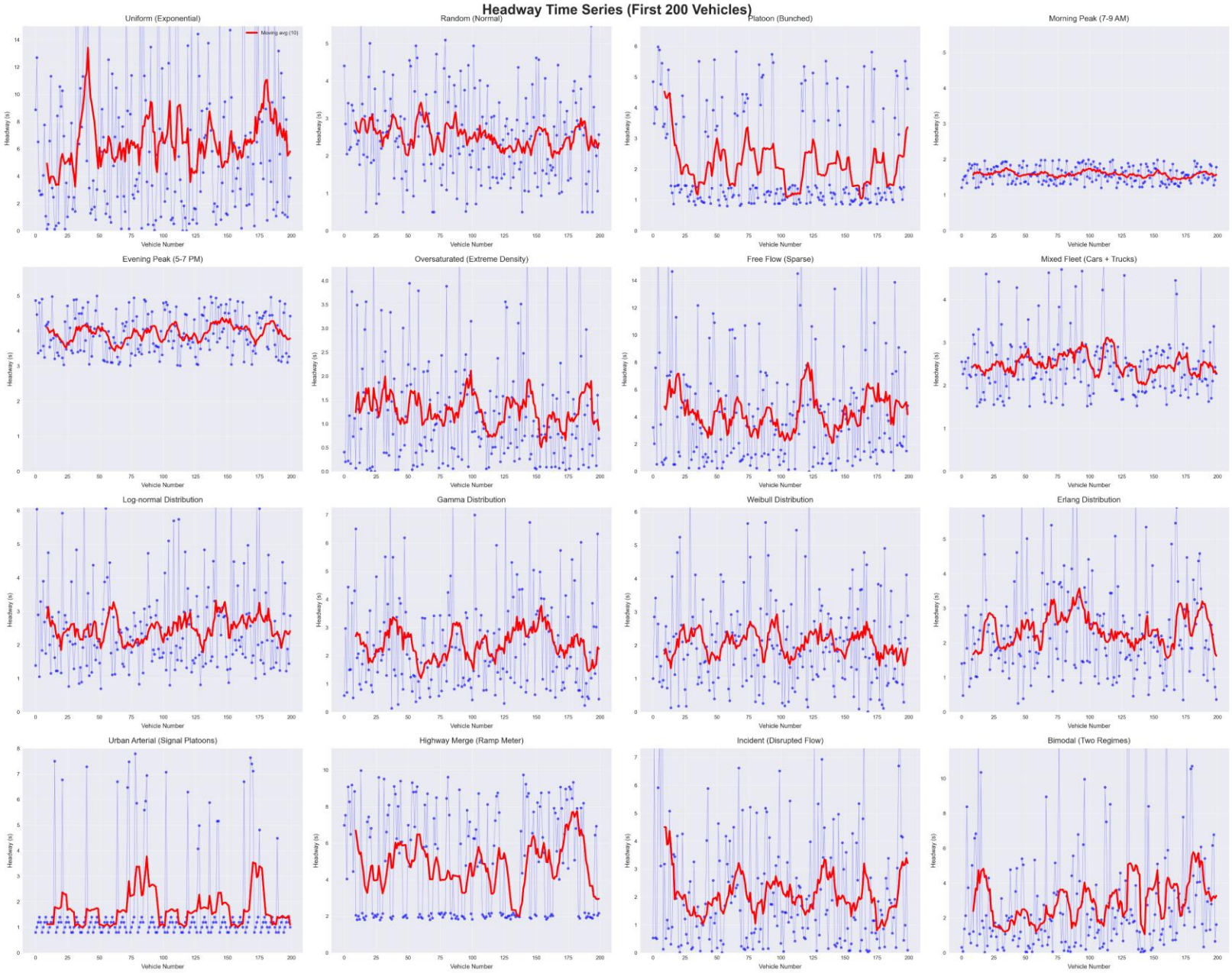
Headway patterns comparison



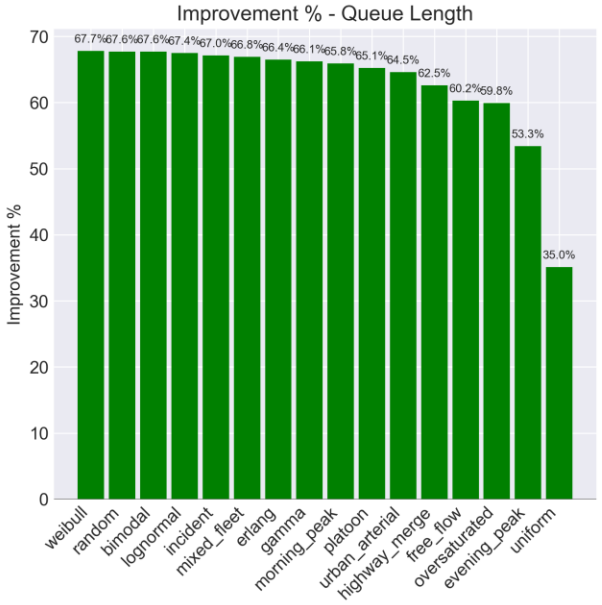


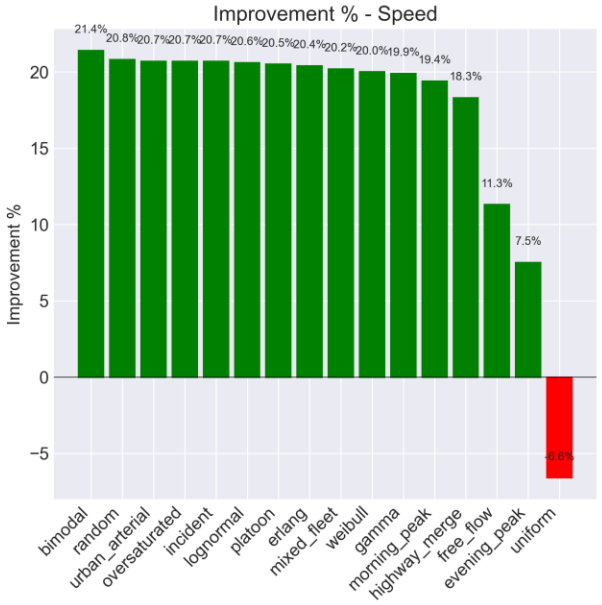
Pattern	Mean	Std	Min	Max	Median
Uniform (Exp	6.02	6.06	0.02	49.03	4.25
Random (Norm	2.54	1.15	0.50	7.21	2.50
Platoon (Bun	2.23	1.64	0.80	6.00	1.31
Morning Peak	2.82	1.09	1.20	5.00	2.68
Evening Peak	3.28	1.21	1.20	5.00	3.61
Oversaturate	1.21	1.17	0.00	9.62	0.88
Free Flow (S	5.02	5.06	0.00	45.70	3.43
Mixed Fleet	2.43	0.69	1.50	4.98	2.35

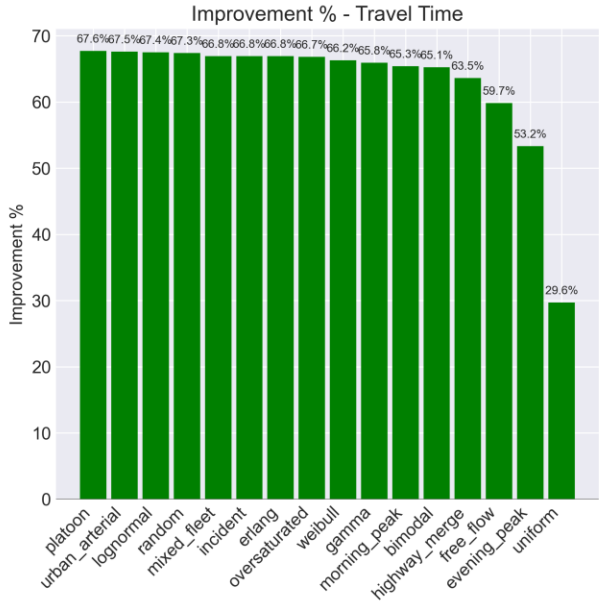
Headway patterns timeseries

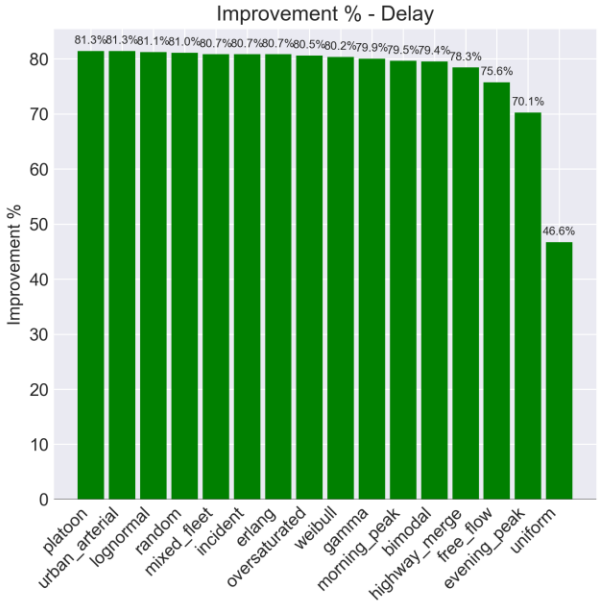


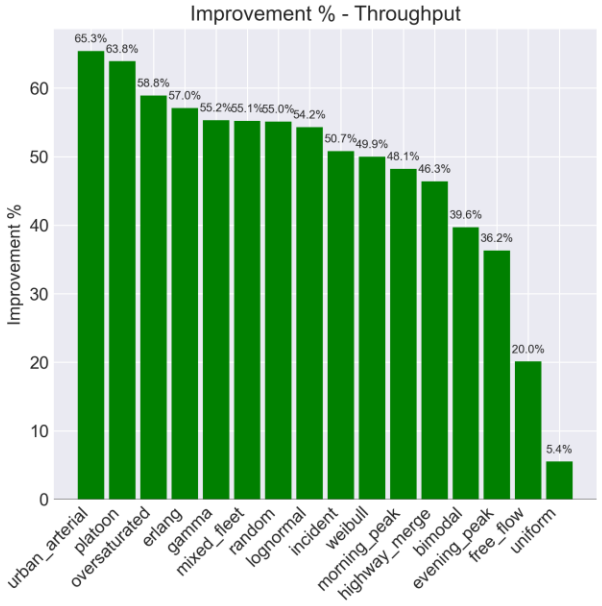
Improvement by metrics

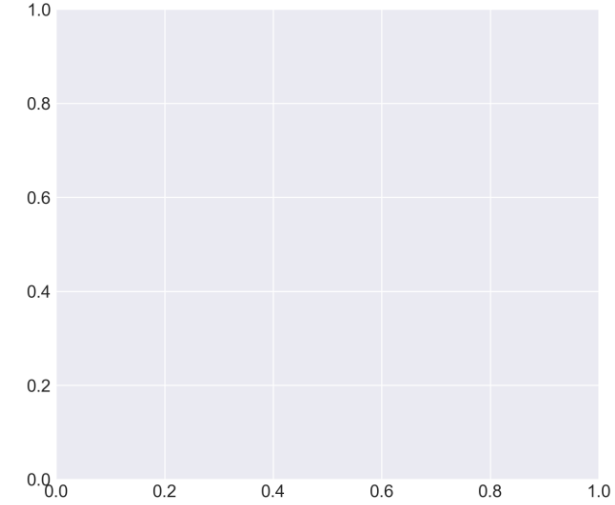




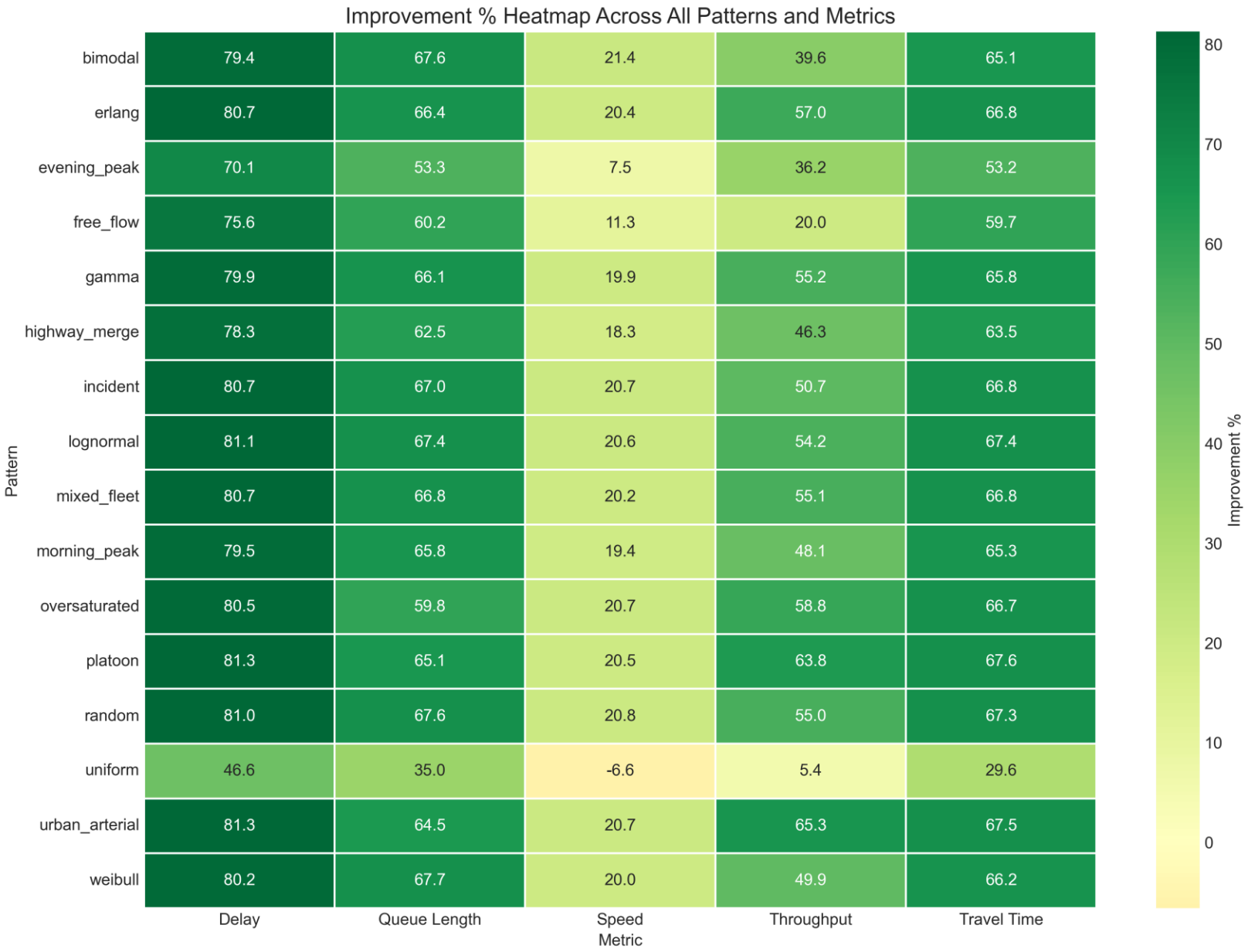






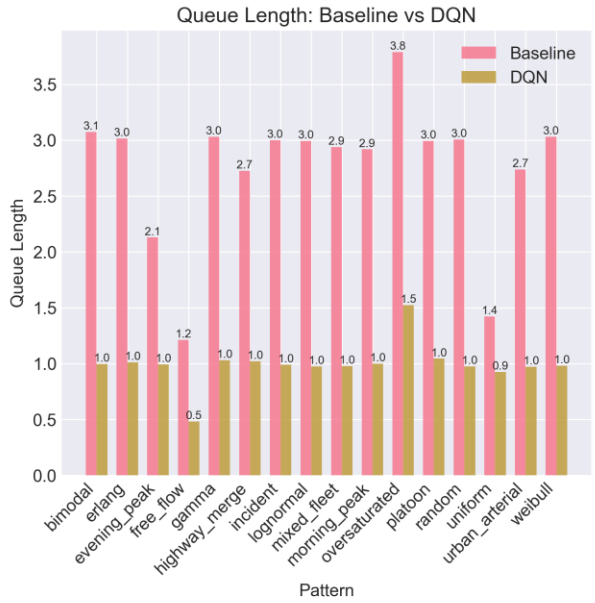


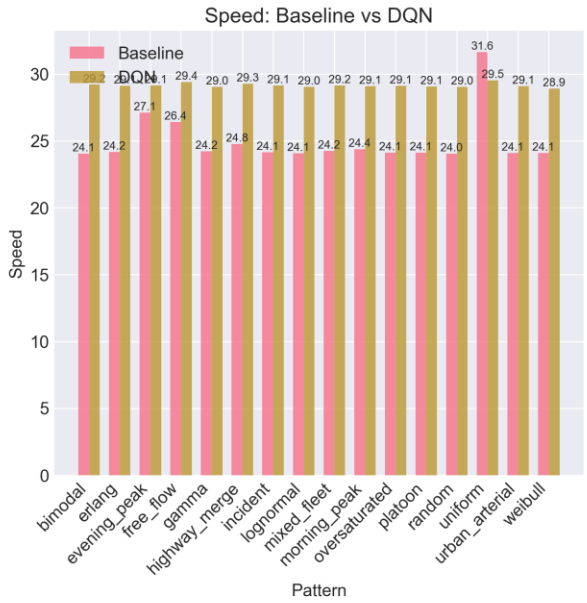
Improvement heatmap

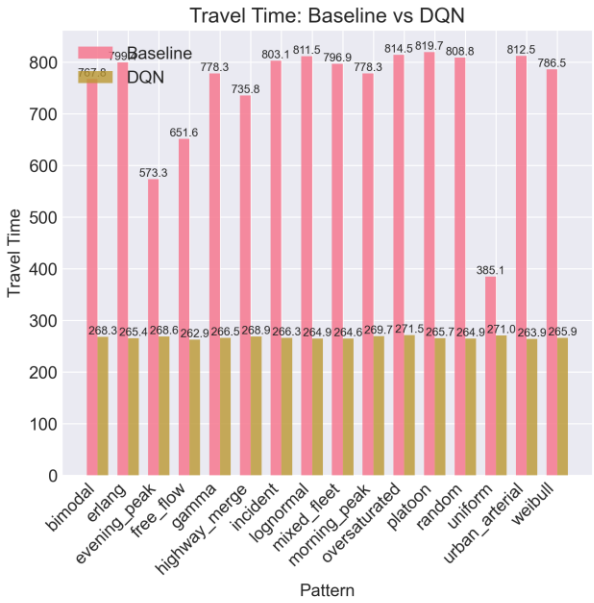


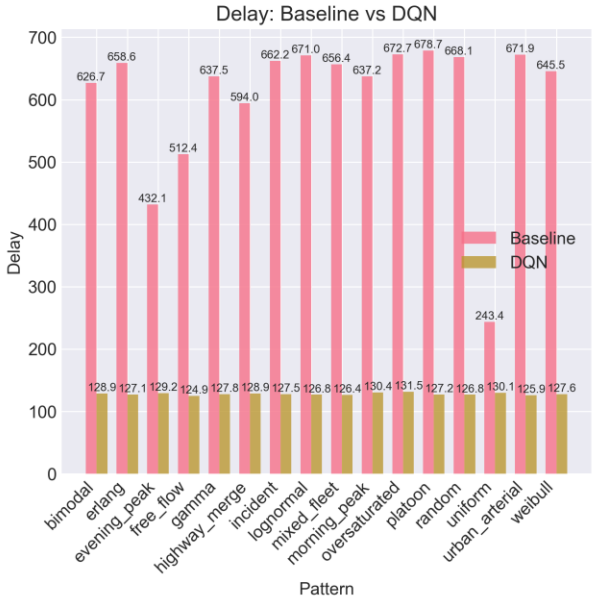
Pattern	Delay	Queue Length	Speed	Throughput	Travel Time
bimodal	79.4	67.6	21.4	39.6	65.1
erlang	80.7	66.4	20.4	57.0	66.8
evening_peak	70.1	53.3	7.5	36.2	53.2
free_flow	75.6	60.2	11.3	20.0	59.7
gamma	79.9	66.1	19.9	55.2	65.8
highway_merge	78.3	62.5	18.3	46.3	63.5
incident	80.7	67.0	20.7	50.7	66.8
lognormal	81.1	67.4	20.6	54.2	67.4
mixed_fleet	80.7	66.8	20.2	55.1	66.8
morning_peak	79.5	65.8	19.4	48.1	65.3
oversaturated	80.5	59.8	20.7	58.8	66.7
platoon	81.3	65.1	20.5	63.8	67.6
random	81.0	67.6	20.8	55.0	67.3
uniform	46.6	35.0	-6.6	5.4	29.6
urban_arterial	81.3	64.5	20.7	65.3	67.5
weibull	80.2	67.7	20.0	49.9	66.2

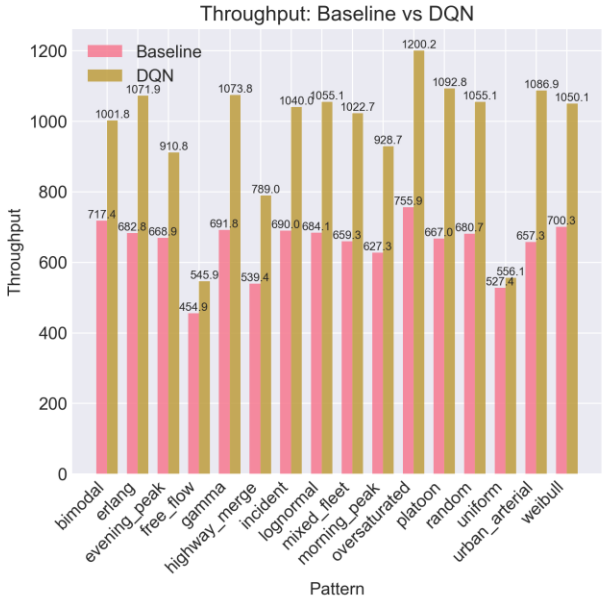
Baseline vs DQN comparison

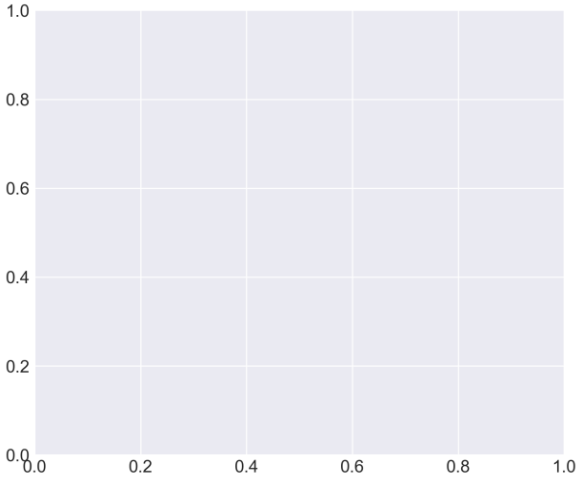




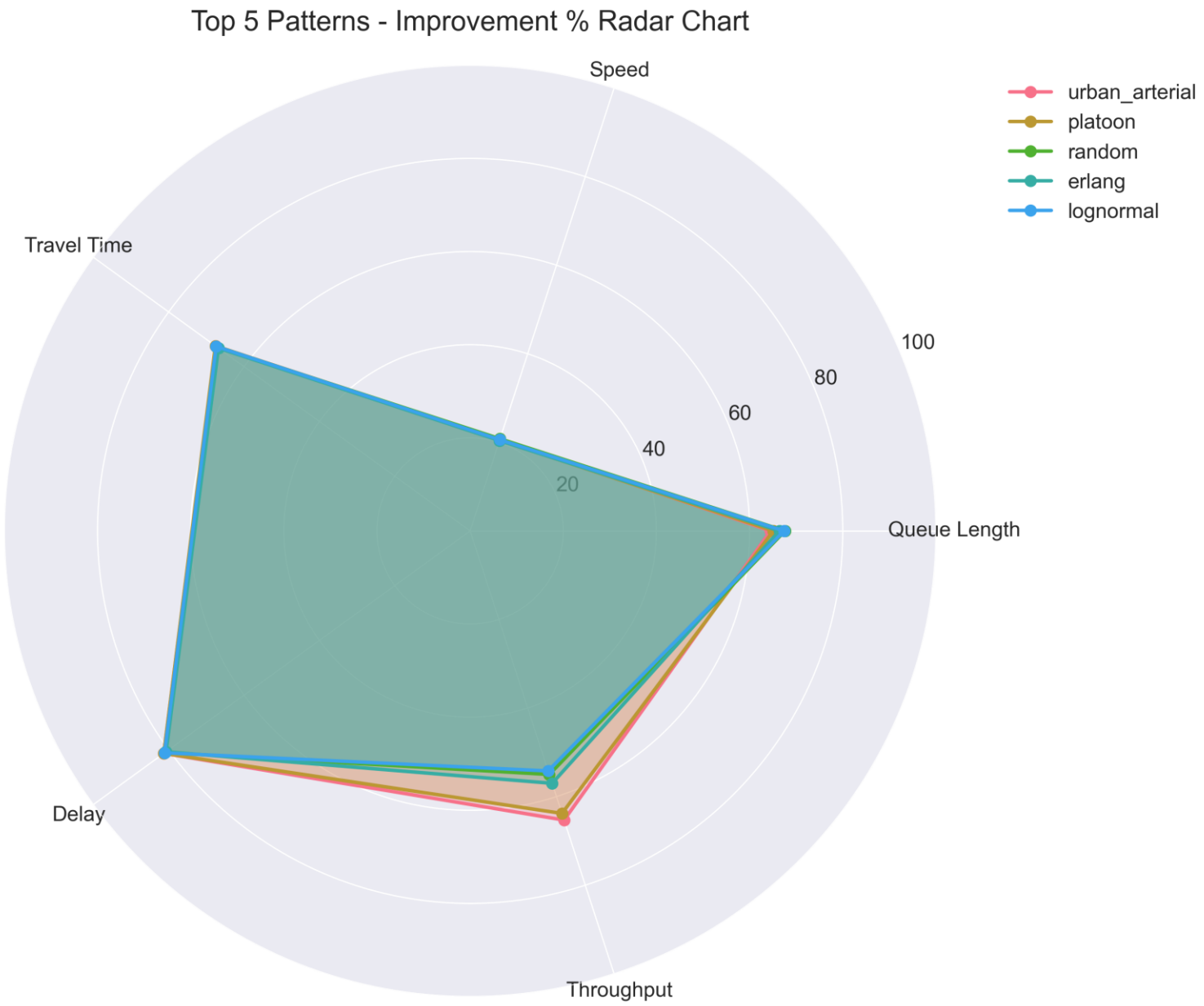




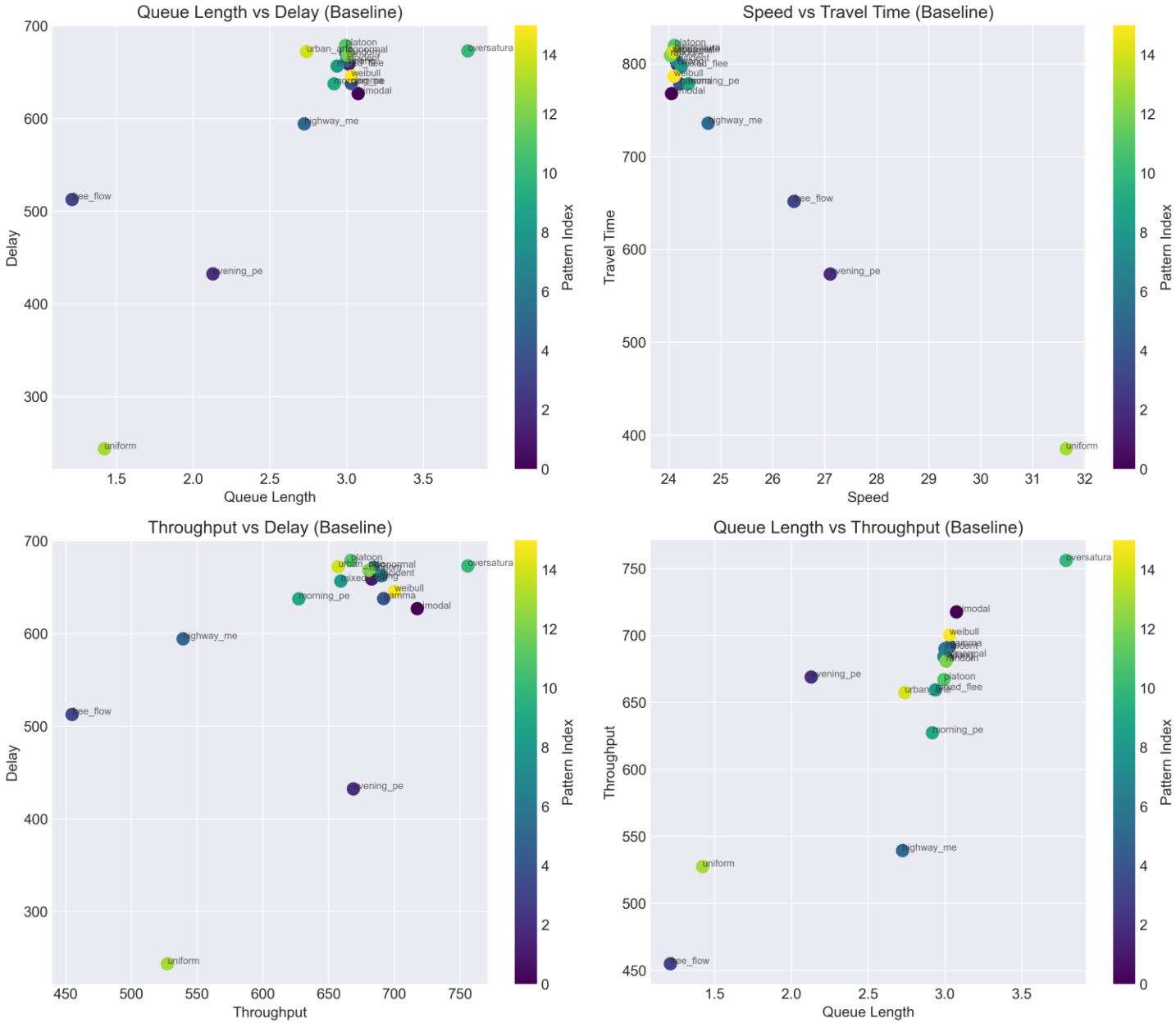




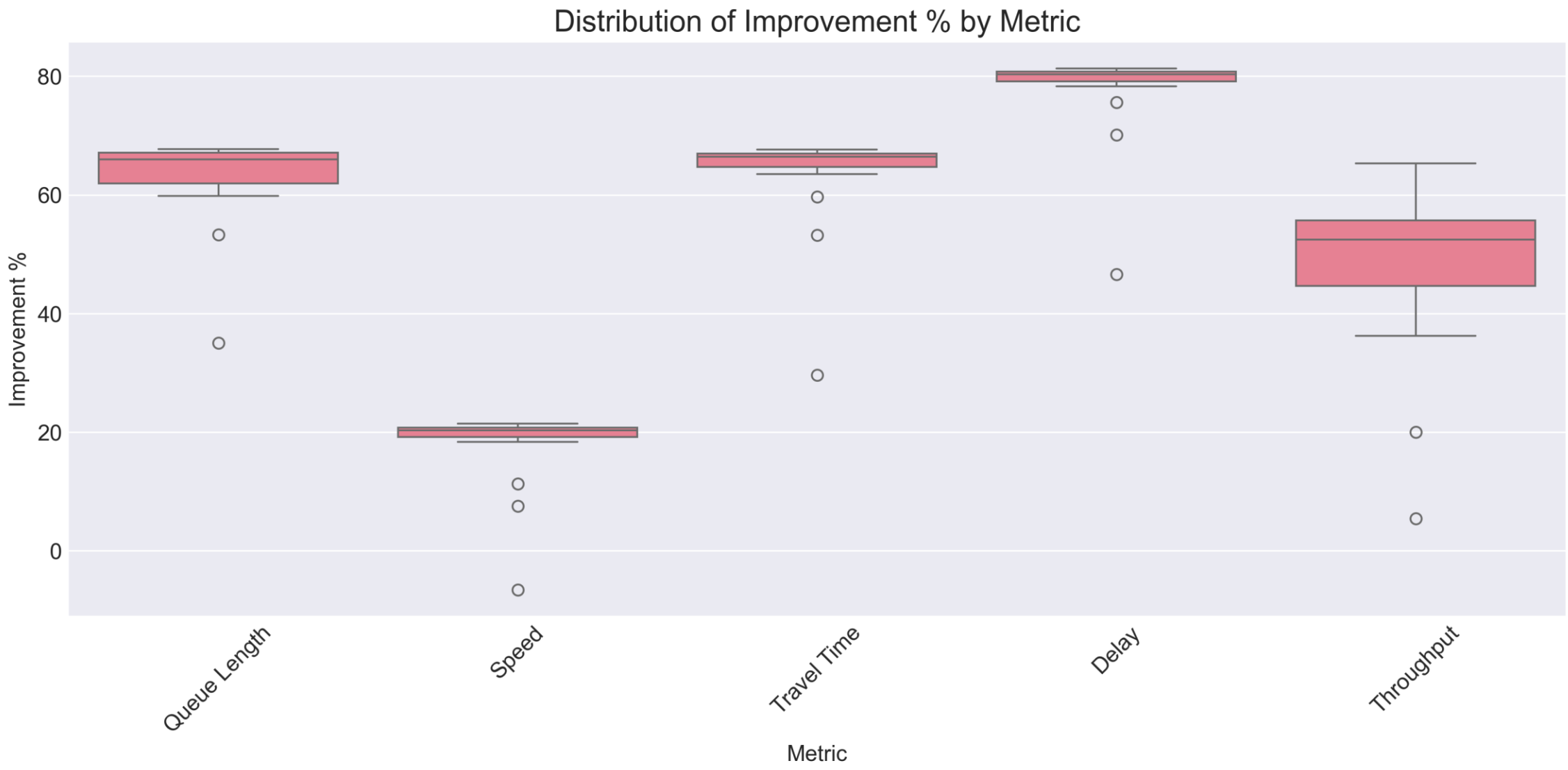
Radar charts for % improvement



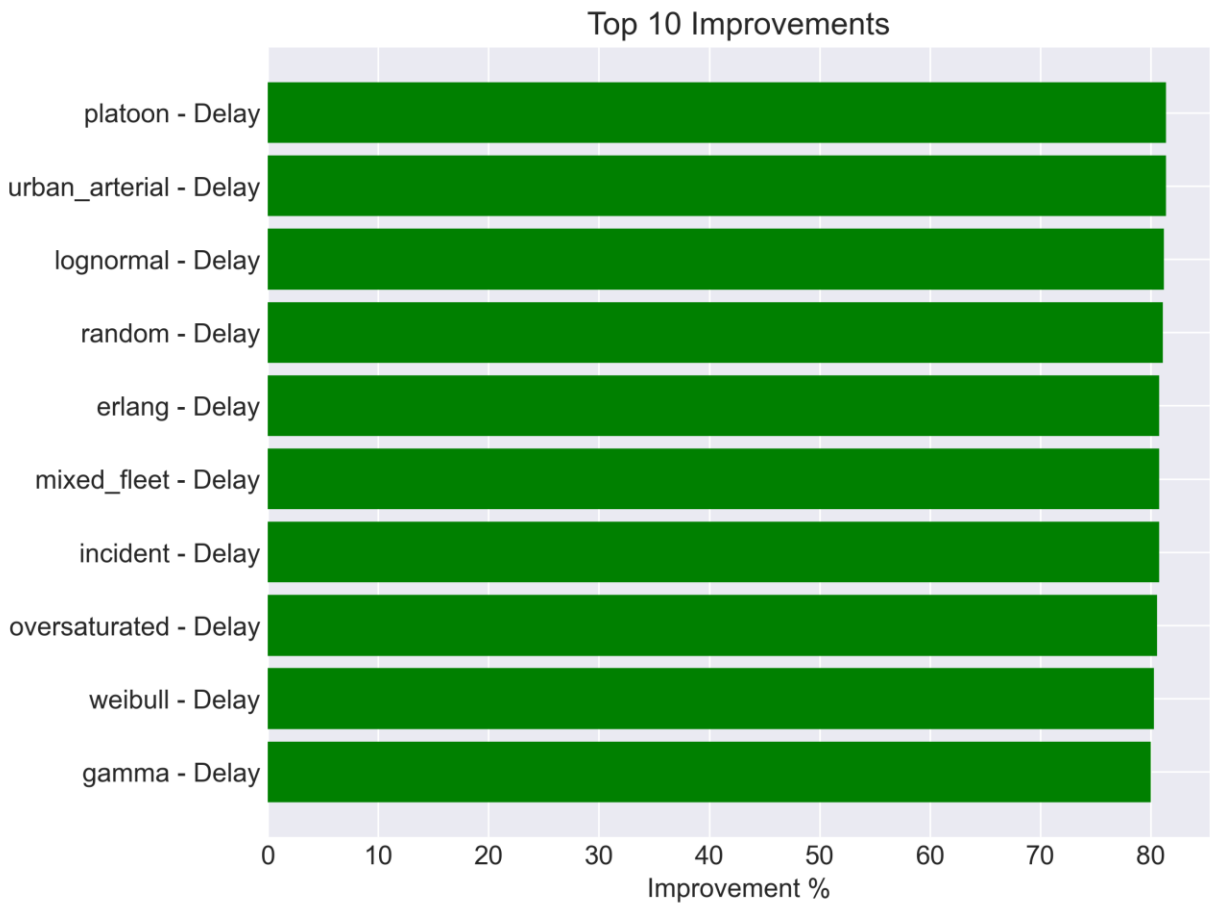
Metric relationship

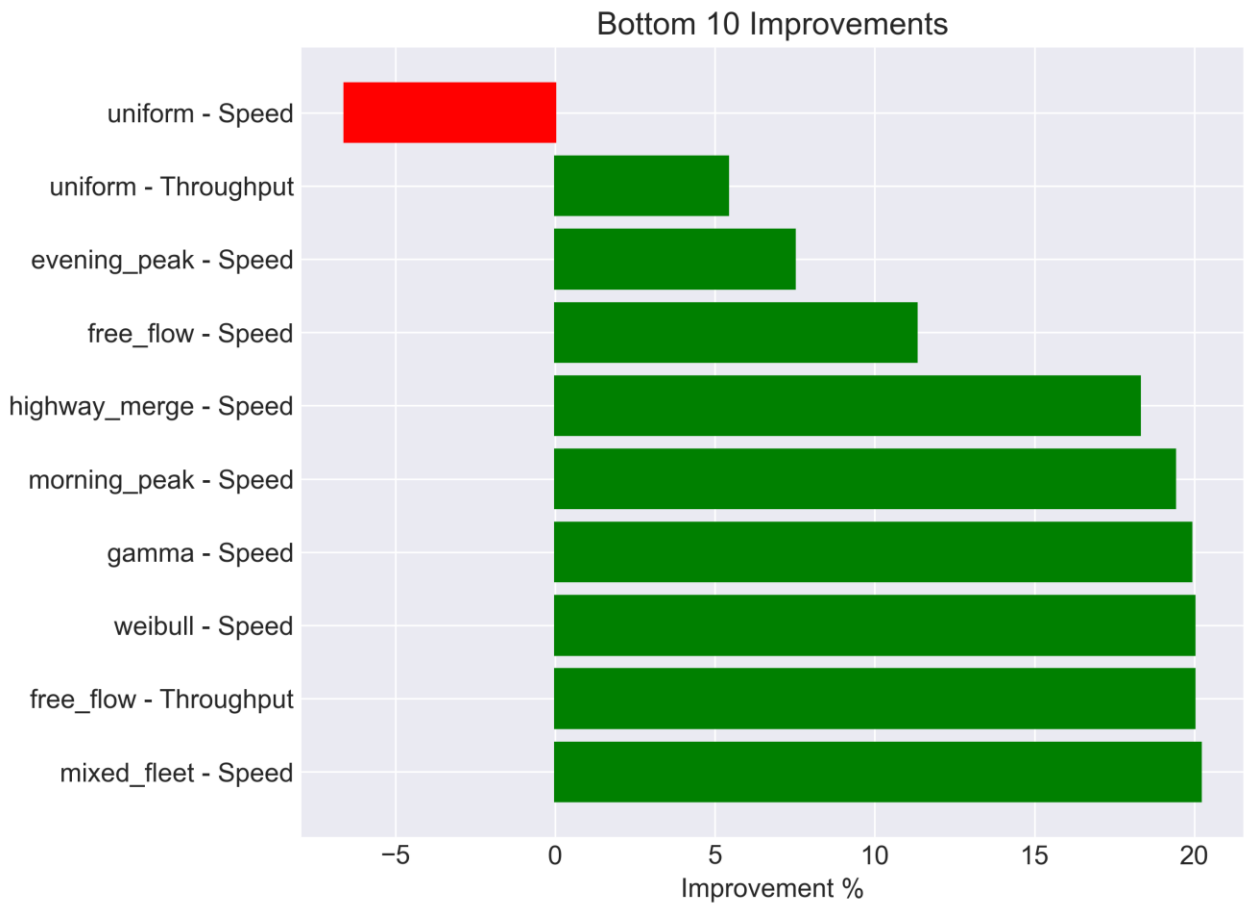


Improvement distribution



Top and bottom 10 improvements





DQN performance analysis

Relevant documentations

GitHub codes: https://github.com/sauravbarua021-eng/DQN_adaptive_signal_performance

Video presentation: https://www.youtube.com/watch?v=yDUcDckrhBI&feature=youtu.be